Project Report: Pattern-Aware Load Shedding for Hot Path Detection in Bike-Sharing Streams

Daniel Granström
Aalto University
Espoo, Finland
daniel.granstrom@aalto.fi

Joonatan Korpela
Aalto University
Espoo, Finland
joonatan.korpela@aalto.fi

Mikael Siidorow
Aalto University
Espoo, Finland
mikael.siidorow@aalto.fi

Abstract

The course project is available at https://github.com/dcgran/CS-E4780-project [2].

This project develops a complex event processing (CEP) system to identify hot paths in the CitiBike data. The goal is to improve bike distribution by detecting where bikes are most often used. The system is built on OpenCEP, which is a unlicensed CEP framework that required several fixes and optimizations to function efficiently. Key improvements include reducing the computational cost of Kleene closure operations and correcting time window handling for trip events with duration.

To manage data processing under heavy load, the system uses a pattern-aware load shedding strategy. Events are prioritized based on their importance to pattern completion and relevance to high-demand stations. Less critical events are dropped when latency exceeds target levels, which keeps the system responsive. The system runs three threads for event feeding, pattern matching, and monitoring, which allows continuous state tracking and low-latency performance.

Performance testing with CitiBike data shows that the system is able to maintain a high recall while reducing latency under load. Results confirm that there is a trade-off between recall and latency but it shows stable memory use and throughput improvements. The optimized design reduces exponential state growth to linear, making real-time pattern detection possible. Limitations include processing on only a single thread and synthetic stress testing, which limits scalability and realism.

Keywords

Complex Event Processing, Load Shedding, Stream Processing, Pattern Detection, Real-time Systems

ACM Reference Format:
Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Project Report: Pattern-Aware Load Shedding for Hot Path Detection in Bike-Sharing Streams. In *Proceedings of CS-E4780 Course Project Evaluation Tables (CS-E4780).* ACM, New York, NY, USA, 3 pages.

1 Introduction

Citibike is a bike sharing program in New York and New Jearsey where users unlock bikes at bike stations and return them at other bike stations[1]. However, in order to keep bikes available at all stations, they need to be moved from areas of high supply to areas of high demand. To find which areas have the highest demand for bikes, it helps to study hot paths, i.e., the paths where there is the most traffic. By identifying these hot paths, we can improve operational efficiency and maintain high availability of bikes.

In this project, we have developed a CEP system for finding such hot paths efficiently using simulated real-time data. The implementation is built on OpenCEP, which is an unlicensed github repository belonging to the user ilya-kolchinsky[3]. OpenCEP is a library and a framework for implementing complex event processing, although it is impossible to install because the project lacks a pyproject.toml and setup.py file. In order to ensure low latency and handle workload bursts, we have implemented a custom pattern-aware load shedding strategy. The complete implementation is available at https://github.com/dcgran/CS-E4780-project.

The project has been implemented using Python and the uv astral package and project manager. The project uses git for version control and vscode as the text editor. To run the project, you must first clone the repository with git and install all required python packages using the command make setup. Then, to run the pattern recognition on test data, you need to run make run–demo.

In this report, we will first cover the general architecture of the system. Then, we will go into more detail of the implementation, such as what improvements were made to the OpenCEP engine and how the pattern-aware load shedding was implemented. Finally, we will evaluate the performance of the system and consider its limitations.

2 System Architecture

The system architecture is built around dynamically adjusting internal parameters to handle resource exhaustion while maintaining low latency. The system reads the state of the CEP engine and makes load shedding decisions based on it.

The system separates event ingestion, pattern processing and system monitoring into separate components. Events are filtered before CEP processing according to a four-tier priority queue. The priorities from highest to lowest are: partial match protection, target station events, noise elimination, and adaptive sampling. System monitoring provides data based on event processing latency which determines parameter adjustment. The system is optimized to maintain low latency rather than high throughput. When latency exceeds targets, sampling rate is lowered to reduce latency. The architecture assumes regular resource constraints.

3 Implementation

3.1 OpenCEP Framework Optimizations

Kleene Closure Performance. The original OpenCEP implementation generates the full powerset (2^n combinations) for Kleene closure operators, which caused an exponential state explosion in

CS-E4780, October 10, 2025, Aalto University
2025. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

our case. With up to 100 partial matches in the hot paths pattern, execution would simply freeze. Our solution pre-filters matches by grouping attributes (bike ID first), then iterates from maximum to minimum size while collecting temporally valid sequences. The complexity drops from $O(2^n)$ to $O(n \cdot m)$ where m is the Kleene size bound. The trade-off is that we sacrifice exhaustive enumeration for linear complexity, essentially prioritizing longest matches over generating all possible combinations.

Time Window Correctness. OpenCEP's Event class only tracked a single timestamp, which breaks time window calculations for events with duration like bike trips. We added an end timestamp getter to the DataFormatter API and modified Event to track both min_timestamp (trip start) and max_timestamp (trip end). This ensures that the 1-hour window evaluation works correctly for trip chains.

3.2 Hot Paths Pattern

The pattern detects bike trip chains ending at NYC's top-3 busiest stations in the August 2018 dataset (402, 435, 497). We use a sequence pattern with three main constraints: same bike ID across all trips in the chain, spatial chaining where each trip's end station matches the next trip's start station, and the final trip must end at one of our target stations. All of this happens within a 1-hour window. Setting the maximum size to 10 on the Kleene closure combined with our greedy matching approach keeps the state explosion under control while still catching all valid hot paths.

3.3 Pattern-Aware Load Shedding

The load shedding uses a four-tier priority hierarchy:

(1) **Partial Match Protection**: Events with bike ID or station ID in active partial matches are never dropped (queried from CEP tree every 0.5s), ensuring that patterns can complete.
(2) **Target Station Events**: Events to/from stations 402, 435, 497 are always kept, biasing toward pattern discovery.
(3) **Same-Station Elimination**: Round trips (start = end station) are dropped as noise.
(4) **Adaptive Sampling**: When latency ratio exceeds 1.5 or queue fill exceeds 70%, remaining events are probabilistically dropped. The sampling rate adapts according to load: the aggressive mode (ratio > 2.0) reduces to 50%, the moderate (ratio > 1.5) to 70%, the relaxed (ratio < 0.5) increases toward 100%.

This ordering prioritizes recall (protecting active patterns) over throughput, trading latency for pattern completeness under load.

3.4 Threading Architecture

Three threads handle different aspects: (1) **Feeder** streams events from CSV, applies load shedding, and queues in a bounded queue (1000 events); (2) **CEP Engine** (main thread) consumes from the queue and performs pattern matching; (3) **Monitor** reports real-time stats without blocking CEP. The bounded queue provides backpressure when CEP cannot keep pace.

Threading is necessary to maintain state continuity across the entire stream. Naive chunked processing loses partial matches between chunks, which drastically reduces recall. Our architecture feeds one continuous stream to preserve all CEP state while enabling asynchronous I/O and monitoring.

4 Performance Evaluation

We evaluate recall, latency, throughput, and scalability using August 2018 CitiBike data (20,000 events sampled from 1.98M events over 31 days). To stress-test load shedding, we use an aggressive 5ms base latency instead of the standard 50ms, forcing the system to shed load even with moderate event rates. A baseline run with no load shedding establishes 100% recall and system capacity. We then test five latency bounds (10%, 30%, 50%, 70%, 90% of baseline latency) where the system adaptively sheds load to meet targets. Metrics tracked include throughput (events/sec), recall (% of baseline matches), memory (MB peak), and drop rate (%).

4.1 Results

Table 1 shows performance across configurations.

Recall vs. Latency Trade-off. Under aggressive latency constraints (5ms base, 10× stricter than standard 50ms), the system demonstrates recall-latency trade-offs. At the 10% bound (0.5ms target), aggressive load shedding drops 43.7% of events, achieving 64.3% recall (117/182 matches). As latency bounds relax, recall improves: 30% bound yields 73.1% recall (133 matches, 40.4% drop), 50% bound reaches the highest recall at 90.7% (165 matches, 35.4% drop), while 70% and 90% bounds show 86.8% and 82.4% recall respectively. The non-monotonic pattern at higher bounds reflects the stochastic nature of adaptive sampling under load—when the system aggressively drops events to meet latency targets, some pattern chains are interrupted unpredictably. This validates that pattern-aware load shedding successfully protects critical events through partial match protection, though extreme latency constraints introduce variability in recall.

Scalability. The baseline processes 20,000 events in 135.0 seconds at 148 events/sec with 41.7 MB peak memory. Under aggressive load shedding (10% bound), throughput increases to 325 events/sec (+120%) by dropping 43.7% of events, achieving 2.2× faster processing at the cost of 35.7% recall loss. At relaxed bounds, throughput decreases to 241-321 events/sec as fewer events are dropped. Memory footprint remains constant at 41.7-41.8 MB across all configurations, demonstrating efficient state management. The system detects trip chains of up to 4 trips within the 1-hour temporal window, validating the temporal filtering and cleanup strategy. OpenCEP's sequential-only evaluation prevents multi-core parallelism, limiting CPU utilization to a single core.

Resource Utilization. CPU usage was dominated by single-core processing due to OpenCEP's sequential-only architecture, with no multi-core parallelization. Under load shedding, CPU remained the bottleneck as event filtering occurs before CEP processing. GPU was not utilized as pattern matching operations are CPU-bound. Memory footprint remained constant at 41.7–41.8 MB across all configurations, demonstrating efficient state management.

Optimization Impact. The Kleene closure optimization ($O(2^n) \rightarrow O(n \cdot m)$) transformed an unusable system (which froze on powerset generation) into one processing 148 events/sec and detecting

Table 1: Performance Evaluation Results (20,000 events, August 2018 CitiBike data, 5ms aggressive base latency)

Config	Events Proc.	Matches Found	Recall (%)	Drop Rate	Throughput (ev/s)	Memory (MB)
Baseline	20000	182	100.0	0%	148	41.7
10% LB	11251	117	64.3	43.7%	325	41.7
30% LB	11927	133	73.1	40.4%	321	41.7
50% LB	12910	165	90.7	35.4%	271	41.7
70% LB	13470	158	86.8	32.6%	257	41.8
90% LB	14719	150	82.4	26.4%	241	41.8

182 patterns. This validates our complexity analysis and demonstrates why algorithmic optimization is critical for practical CEP applications.

4.2 Limitations

Dataset Scale. The August 2018 dataset contains approximately 1.98M events over 31 days. While the natural event rate is low, we artificially stress-tested the system using aggressive 5ms base latency (10× stricter than standard 50ms). This successfully triggered load shedding across all latency bounds, with drop rates ranging from 26.4% to 43.7%. However, this artificial constraint limits real-world applicability—production systems would likely use higher latency targets. Future work should evaluate with time-compressed replay at realistic latency bounds to better understand production performance.

OpenCEP Constraints. Sequential-only evaluation limits scalability. The lack of a public API for partial match state required fragile internal tree traversal. These constraints reduce portability and prevent multi-core scaling.

5 Conclusion

This project shows that a complex event processing system can identify high-demand routes in CitiBike data in real time. By optimizing the OpenCEP framework, the system runs efficiently without freezing under heavy workloads. The new approach to handling Kleene closures reduces computation from exponential to linear time, which makes pattern detection possible.

The pattern-aware load shedding strategy keeps latency of the system low while preventing important events from being dropped. It balances recall and speed by dropping less relevant data when the system is under load. Performance testing confirms that recall decreases as the desired latency is decreased, but the same patterns are still detected. The main limitations of the system are the single-threaded execution of the OpenCEP engine and the use of simulated stress conditions. These restrict the system's scalability and they do not fully reflect real-world conditions.

Team Contributions

Individual Contributions. The team organized weekly working sessions every Monday (2-3 hours) where we ideated the solution together. Granström set up the initial project structure. Siidorow imported the OpenCEP library and fixed critical bugs in the Kleene closure implementation and time window handling. Siidorow also implemented the baseline system. Granström enhanced the implementation and built the benchmarking suite. Korpela handled communications with course staff. All team members provided continuous feedback throughout development and contributed to writing the report.

Design Decisions and Challenges. The most important decisions our group made about the system design includes what load-shedding methods should be implemented and how partial matches are stored and prioritized. OpenCEP had many bottlenecks but we were able to resolve those with the multi-threaded event feeder design. These decisions were discussed and made in our weekly working sessions.

References

[1] Citi Bike. 2025. *How it Works — Citi Bike NYC.* https://citibikenyc.com/how-it-works Accessed: 2025-10-10.
[2] Daniel Granström, Joonatan Korpela, and Mikael Siidorow. 2025. Pattern-Aware Load Shedding for Hot Path Detection in Bike-Sharing Streams. https://github.com/dcgran/CS-E4780-project. Accessed: 2025-10-10.
[3] Ilya Kolchinsky. 2025. OpenCEP: A highly efficient and expressive CEP framework in Python. https://github.com/ilya-kolchinsky/OpenCEP. Accessed: 2025-10-10.